Banking Churn Analysis & Modeling

A code-free, step-by-step student guide for the tabular machine-learning notebook

Dataset source
Use the original Kaggle dataset: Kaggle - Churn Modelling
Dataset owner: shrutimechlearn | File used by the notebook: Churn_Modelling.csv

What students are building

The notebook is a binary classification project for a bank. Students use customer information to predict the target **Exited**: whether a customer is likely to leave the bank. The project has two goals: understand the drivers of churn and compare several predictive models.

Expected deliverables

A clean, reproducible notebook that explains the problem, data, decisions, results, and final recommendation.

Charts that describe the customer population and show relationships with churn.

A comparison of Logistic Regression, Support Vector Classifier, Decision Tree, Random Forest, and XGBoost.

A saved final model plus the fitted preprocessing artifacts and final feature list for later deployment.

Workflow at a glance

1. Frame	Define churn and the business decision.
2. Understand	Inspect records, fields, quality, and distributions.
3. Prepare	Select features, split data, encode, scale, and handle imbalance.
4. Model	Train and tune several classifiers.
5. Decide	Evaluate churn detection, select a model, and prepare deployment assets.

Step-by-step notebook plan

Step 1 - State the problem

Explain that churn means a customer leaves the bank. Define the prediction target as Exited and state why early identification lets the bank prioritize retention actions.

Step 2 - Set up the notebook

Organize the notebook into clear sections: imports, data understanding, data quality, EDA, preprocessing, modeling, evaluation, and deployment. Record package versions and use a fixed random seed so the work can be reproduced.

Step 3 - Load the dataset

Download Churn_Modelling.csv from the Kaggle link on page 1, place it in the project data folder, and load it as a table. Display a small sample to confirm the file, rows, columns, and target are present.

Step 4 - Build a data dictionary

Describe each field in plain language. Highlight CustomerId, RowNumber, and Surname as identifiers rather than useful predictive signals. Identify Geography and Gender as categorical variables, numeric fields such as Age and Balance, and Exited as the binary target.

Step 5 - Audit data quality

Check the table shape, data types, duplicated rows, missing values, and improbable values. Use a concise note to document what was found and whether a cleaning action was needed.

Step 6 - Inspect potential outliers

Use boxplots for CreditScore, Age, Balance, and EstimatedSalary. Outliers are not automatically errors; decide whether they are plausible banking observations before changing or removing any values.

Step 7 - Explore the data

Perform exploratory data analysis before modeling. The purpose is to understand who the customers are, how common churn is, and which patterns deserve further investigation.

- Review the distribution of CreditScore to understand its range, shape, and whether extreme values are common.
- Count and visualize customer Geography and Gender to understand population composition.
- Inspect NumOfProducts because product holdings may be strongly connected to customer behavior.
- Create a correlation heatmap for numeric features and Exited. Treat correlation as an initial clue, not proof of causation.
- Add target-focused comparisons, such as churn rate by geography, gender, product count, activity status, and age band. These make the business story stronger than distribution charts alone.

Step 8 - Report EDA findings

Under every important chart, write one short interpretation in business language. For example, explain whether a segment has a higher churn rate, then identify a question the bank should investigate. Do not claim that a feature causes churn unless the analysis supports that conclusion.

Preparing data for reliable modeling

Step 9 - Remove non-predictive identifiers

Exclude RowNumber, CustomerId, and Surname from the modeling features. They describe record identity, not customer behavior, and can create misleading patterns.

Step 10 - Separate features and target

Create the feature matrix from the remaining input columns and keep Exited separately as the target label. Verify that the target is not accidentally included among the inputs.

Step 11 - Split before learning from data

Create a training set and a held-out test set, preserving the churn proportion when possible. The test set must remain unseen until the final evaluation, so it represents future customers.

Step 12 - Encode categorical variables

Convert Geography and Gender into numeric indicator features. Preserve the resulting feature names and column order because the deployment app must apply exactly the same transformation to new customer records.

Step 13 - Scale numeric variables

Standardize the selected numeric columns, including CreditScore, Age, Tenure, Balance, and EstimatedSalary. Scaling is especially important for distance- and margin-based models such as SVC. Fit the scaler only on training data, then use it to transform both training and test data.

Step 14 - Address class imbalance

Measure the share of churners versus non-churners. If using SMOTE or another resampling method, apply it only to the training data after the split. This prevents synthetic information from leaking into test evaluation.

> **Important reliability rule**
> The reference notebook scales before splitting and tunes SVC on the full dataset. In a student submission, split first, then fit preprocessing, resampling, and hyperparameter search on training data only. This avoids data leakage.

Train and tune the models

Use the same prepared training data for each model so the comparison is fair. Record settings, validation results, test score, and churn-focused metrics.

Model	Why include it	Student task
Logistic Regression	Simple, interpretable baseline.	Train first and use it as a reference point.
Support Vector Classifier	Can capture non-linear decision boundaries.	Tune kernel and regularization with cross-validation on training data only.
Decision Tree	Easy to explain, but can overfit.	Control depth and purity rules; compare train and test behavior.
Random Forest	Ensemble of trees; robust non-linear baseline.	Inspect whether it detects churners, not only overall accuracy.
XGBoost	Gradient boosting often performs strongly on tabular data.	Tune cautiously, then explain why it is or is not the final choice.

Evaluate, select, and package the result

Step 15 - Generate test predictions

After choosing each trained model, predict the held-out test set. Use the same test set for every candidate model and do not use the results to repeatedly retune parameters.

Step 16 - Evaluate churn detection

Calculate accuracy, precision, recall, and F1-score. Draw a confusion matrix for each model so students can see true positives, false positives, true negatives, and false negatives.

Step 17 - Choose the business metric

Explain that accuracy can be misleading when churners are a minority. Recall answers how many real churners were found; precision answers how often a churn alert is correct. Select the trade-off that fits the bank's retention budget and risk.

Step 18 - Compare models clearly

Create one comparison table with each model's accuracy, precision, recall, and F1-score. Discuss both strengths and limitations. The reference notebook favors XGBoost for a balanced precision-recall result, while SVC has stronger precision.

Step 19 - Select the final model

Choose the model based on the agreed business objective, not simply the highest accuracy. State the reason in one paragraph and mention which churn errors remain costly.

Step 20 - Save deployment artifacts

Save the selected model, the fitted scaler, and the final feature-column list. These artifacts ensure a future application processes new customer data in exactly the same way used during training.

Final submission checklist

Check	What good work looks like
Data source	Original Kaggle link is cited and the correct CSV is used.
Analysis	Charts have concise interpretations tied to churn, not just visual output.
Validity	Split occurs before fitting scaler, resampling, or tuning; test set remains untouched.
Models	All five candidate models are assessed consistently.
Decision	Final model rationale considers precision, recall, and the bank's retention objective.
Deployment	Model, scaler, and feature schema are retained together.

Source note: The dataset URL was verified against the notebook's Kaggle metadata (dataset ID 156197) and the notebook's input filename, Churn_Modelling.csv.